AppSec Engineer Practical Guide 2026

AppSec Deep-Dive — Every Task as a Practice Routine

- Task 1 — Morning triage: SAST/SCA results in Jira + dev Slack pings
- Task 2 — Threat modeling (STRIDE) for new features
- Task 3 — Secure-by-default library for developers
- Task 4 — Custom Semgrep rules (the tooling differentiator)
- Task 5 — Weekly rhythm artifacts
- Task 6 — Monthly metrics to a CISO (practice this format)

30-Day AppSec Skill Sprint (start Monday)

- The defensiveness rule (your communication superpower)

AppSec Deep-Dive — Every Task as a Practice Routine

Companion to Security Engineer Field Guide 2026 · Kelvin Odarikpe

> This document takes the real AppSec task list and turns **each task into something you can practice, document, and ship as evidence**. For every task: what it looks like in a real job, how to practice it today (SOP + commands), and what artifact it produces for GitHub/LinkedIn.

Task 1 — Morning triage: SAST/SCA results in Jira + dev Slack pings

What the job looks like: overnight scans (Semgrep/Trivy/Dep-Check) drop findings into Jira; devs ping “why is my PR failing?” in Slack. Your job: triage fast, triage accurately, never make the dev feel stupid.

Practice SOP (build this as a mini-lab): 1. Create a repo with an intentionally vulnerable Flask/Node app (your `01-devsecops-pipeline`). 2. Wire Semgrep + pip-audit + Gitleaks into GitHub Actions so findings auto-create Jira-style issues (use the GitHub Issues API or a scheduled job). 3. Write a **triage decision tree** (this is the interview artifact):

```
Finding arrives
├─ Is it in code we ship? (check path/ownership) → no: reassign/close with comment
├─ Is it reachable? (is the vulnerable function called from an exposed route?)
│    ├─ Yes → Critical/High, reproduce with a PoC (curl/pwntools), file with fix
│    └─ No → Medium/Low, ticket with "defense in depth" tag
├─ Is it a known FP pattern? → suppress with justification comment in the rule, not in a doc
└─ Not sure → 15-minute timer; if still unsure, escalate with your analysis
```

1. Write a **triage decision doc** (1 page): the 5 questions you ask for every SAST finding — reachable? exploitable? input validated upstream? compensating control? fix cost?

Evidence to ship: a `docs/triage-decision-tree.png` + 3 example findings with your reasoning. This alone demonstrates the “explain without making devs defensive” skill — which sources say is the real differentiator.

Commands to get fluent in (daily reps):

```
semgrep --config=p/ci --config=p/security-audit --error .       # SAST gate
pip-audit -r requirements.txt                                   # SCA
trivy fs --scanners vuln,secret,misconfig .                     # all-in-one
trivy image --severity HIGH,CRITICAL --exit-code 1 app:latest
grep -rn "password\s*=" --include="*.py" .   # poor-man's secret scan (know why tools beat grep)
```

Task 2 — Threat modeling (STRIDE) for new features

Practice SOP (do this weekly on a real feature — even a made-up one): 1. Pick a feature (e.g., "add SSO login to vulnbank"). 2. Draw the data-flow diagram: users → auth → API → DB → external SaaS. 3. Walk STRIDE element by element:

Element	Question	Example finding (vulnbank SSO)
Spoofing	Can an attacker impersonate a user/IDP?	Unsigned JWT accepted → forge admin token
Tampering	Can data be modified in transit/at rest?	No signature on the JWT payload → role escalation
Repudiation	Can actions be denied?	No audit log on login → who did what, unprovable
Info disclosure	Can data leak?	Token in URL → leaks via Referer
Denial of Service	Can the flow be exhausted?	No rate limit on /login → credential stuffing
Elevation of Privilege	Can a user become admin?	`/admin` route has no authorization check

1. Write the **threat model doc**: diagram + table + top-3 mitigations with effort estimates.
2. Ship it as `docs/threat-model-sso.md` in the vulnbank repo — with the mitigations implemented as commits.

Interview phrasing: "I ran a STRIDE on the SSO flow, found unsigned-JWT acceptance and missing login rate limiting, and wrote the mitigations as user stories so the devs could sprint them."

Task 3 — Secure-by-default library for developers

Goal: a small Go or Python library that makes the secure path the easy path. This is the highest-leverage AppSec artifact — it shows you build for developers, not at them.

SOP — build `securelib` (Python):

```
mkdir securelib && cd securelib && git init
# Contents: safe_db(), safe_jwt(), rate_limit(), safe_redirect()
```

securelib/db.py — parameterized-by-default wrapper:

```python
"""Safe-by-default DB access: parameterized queries only, no string formatting."""
import sqlite3
from contextlib import contextmanager

class SafeDB:
    def __init__(self, path):
        self.conn = sqlite3.connect(path, check_same_thread=False)

    def query(self, sql: str, params: tuple = ()):
        """Rejects queries that look string-formatted (defense in depth)."""
        if any(c in sql for c in ("{}", "%s", "+")):
            raise ValueError("Interpolated SQL is forbidden — use ? placeholders")
        return self.conn.execute(sql, params).fetchall()

# usage: db.query("SELECT * FROM users WHERE name=?", (user,))
```

securelib/auth.py — fail-closed JWT check:

```python
import jwt  # pyjwt

def verify(token: str, secret: str, required_role: str | None = None) -> dict:
    """Verify JWT; reject 'none' algorithm; enforce role. Raises on any problem."""
    try:
        claims = jwt.decode(token, secret, algorithms=["HS256"])  # alg pinning
    except jwt.InvalidTokenError as e:
        raise PermissionError(f"invalid token: {e}") from None
    if required_role and claims.get("role") != required_role:
        raise PermissionError(f"role '{claims.get('role')}' cannot access this")
    return claims
```

Evidence: unit tests + a README that reads like documentation devs would actually use. Add a CI workflow that runs *your own tool* on itself (self-hosting demo).

Task 4 — Custom Semgrep rules (the tooling differentiator)

SOP — write 5 rules for your stack, each with a true + false-positive test:

```yaml
# rules/jwt-none-alg.yaml
rules:
  - id: jwt-none-algorithm
    message: >
      JWT decoded with 'none' algorithm — signature verification bypass (CWE-347).
    severity: ERROR
    languages: [python]
    patterns:
      - pattern: jwt.decode($TOK, ..., algorithms=[...])
      - metavariable-regex:
          metavariable: $TOK
          pattern: .*  # plus a pattern-either for the algorithms kwarg
    metadata:
      cwe: "CWE-327"
      owasp: "A02"
```

Then a **rule catalog doc**: rule ID → what it catches → known FP → how to suppress safely. Publish as its own repo — this doubles as your Security Champions starter kit (3 rules + triage guide + 10-slide training deck).

Task 5 — Weekly rhythm artifacts

Weekly activity	Evidence artifact (what you publish)
Security Champions sync	1-page agenda + 1 takeaway doc in your repo
Design review	Checklist doc (authn/authz, input handling, secrets, logging, rate limits, PII flow)
Vuln backlog grooming	Metrics table: open criticals, age, SLA compliance
Bug bounty triage	1 anonymized write-up per month: "What a valid report looks like"
Training session	10-slide deck (e.g., "Top 5 Flask vulns and how our CI catches them")

Task 6 — Monthly metrics to a CISO (practice this format)

> APSEC METRICS — July 2026
> Open criticals: 3 (was 8) | Median time-to-fix: 4d (was 11d)
> Fix-SLA compliance: 92% critical / 85% high
> SAST FP rate: 31% → 18% (custom rules + suppression policy)
> Threat models completed: 3 | Security Champions active: 5/8 teams
> Bug bounty: 12 reports, 4 valid, $0 duplicates, avg triage 36h
> Next: pipe DAST results into Jira automatically; onboard 2 more champions

---

30-Day AppSec Skill Sprint (start Monday)

Week	Theme	Daily reps (45–90 min)	Weekend project
1	SAST/SCA depth	1 Semgrep rule/day for your stack; run Semgrep+pip-audit on 3 OSS repos	Project 1 (pipeline)
2	Threat modeling	STRIDE one feature/day (pick apps: OWASP crAPI, Juice Shop)	vulnbank threat model doc + mitigations shipped
3	DAST + pentest craft	ZAP baseline → manual session/IDOR testing on Juice Shop	Pentest report (≤5 findings, dev-ready fixes)
4	Program building	Draft Security Champions kit + monthly metrics template	Publish kit + a "week in the life" LinkedIn post

Daily learning stack (30 min): OWASP Top 10 → 1 CWE deep-dive → 1 real advisory (GitHub Security Advisories feed) → write 1 custom rule or 1 finding note. By day 30 you have: 1 pipeline repo, 1 threat-model doc, 1 pentest report, 1 champions kit, 4 LinkedIn posts — that's a complete AppSec interview portfolio.

The defensiveness rule (your communication superpower)

Every finding message you send ends with: *"happy to pair on the fix if useful"* — and every design-review verdict offers at least one alternative path, not just "no". The phrase that works, from teams that do this well: **"here's the smallest change that ships your feature and closes the risk."** Practice writing 5 such messages before your next interview; this is what senior panels probe with "tell me about a time you disagreed with a developer."